

操作系统A

■ 可持久化的操作系统核心模拟器设计与实现



北京林业大学



主讲教师：李巨虎 / 孟伟 / 范新



适用班级：信息学院各专业



日期：2026年6月

我们要做的是什​​么？—— 设计目标与核心功能

核心目标：设计并实现一个交互式的操作系统核心模拟器，通过模拟底层运行机制，完整复现操作系统对进程生命周期、调度策略及内存资源的管理逻辑，让抽象的内核原理可视化、可交互。

进程管理模块

构建进程控制块(PCB)结构与进程树，精准模拟进程在创建、就绪、运行、阻塞到终止的全生命周期状态转换，还原OS对基础执行单元的底层管理逻辑。

进程调度模块

实现经典的多级反馈队列(MLFQ)调度算法，支持自动时序执行与单步调试两种模式。直观展示不同优先级进程的抢占、轮转与时间片分配过程，体现调度的公平性与高效性。

内存管理模块

模拟动态分区内存分配机制，支持首次适应等经典算法。实现内存块的申请、释放操作，并自动执行内存碎片紧缩，动态可视化内存资源的分配状态与空间变化。

核心特性：状态可持久化

系统运行状态支持二进制序列化存储，断点退出后重启可无缝恢复现场，实现“存档读档”式的连续模拟体验。

核心特性：多线程架构

前台交互线程响应用户指令，后台维护线程异步处理核心调度逻辑，有效解耦UI操作与内核计算，提升系统的并发处理能力。

核心特性：多实例共享

允许多个客户端窗口并发连接至同一内核状态，支持协作式模拟分析或教学场景下的同步视角观察。

可执行程序：编译后的命令行工具，具备全功能交互与参数配置能力。

课程设计报告：含需求分析、架构设计、算法实现及完整的测试结果分析。

工程源代码：模块解耦、注释规范，包含核心算法与完整的测试用例集。

系统总体架构——给老师看的交互模式

前台用户交互层

多终端并发接入
每个用户对应独立交互线程
读取命令并封装消息入队

共享消息队列

解耦前后台执行逻辑
异步化处理用户请求
削峰填谷，缓冲高并发压力

后台维护线程

单例守护进程
持续消费队列任务
执行内核调度与状态变更

持久化存储层

内存共享数据结构
(PCB表/内存分区表)
定期快照写入二进制文件

前台交互：快速响应与投递

作为系统的“入口”，前台线程不做复杂计算，仅负责读取用户在终端输入的操作指令。将指令标准化为消息对象后，立即投递至共享队列并返回，保证用户操作的即时反馈体验。

后台执行：核心逻辑驱动

后台线程是真正的“大脑”。它独立循环监听队列，取出任务后执行如进程创建、内存分配等核心内核操作。执行完成后，不仅要更新内存中的系统状态，还要负责将关键数据定期持久化到物理磁盘。

数据层：临界区安全防护

共享数据结构是系统状态的载体。所有线程在访问（读/写）PCB表或内存分区表时，必须严格通过互斥锁（Mutex）进行临界区保护，杜绝并发修改带来的数据不一致问题。

核心技术难点：跨实例并发共享

突破单进程地址空间限制，实现多个独立命令行客户端（实例）对同一份内核状态的共享。需设计高效的跨线程/跨实例通信机制，确保数据同步的实时性。

老师现场验收核心标准

双窗口联动测试：开启两个终端实例，在A实例创建进程后，B实例刷新必须立即显示新进程；反之亦然。操作过程中系统不能出现死锁、数据错乱或无响应。

进程管理——你要实现什么？

【验收清单】共需实现10个核心命令（每题2分，合计20分），需严格按预期效果向老师演示功能执行结果。

基础控制组：

1. create_pcb：新建进程后，用show_pcb核对PID、状态、运行时间等字段准确无误。
2. kill_pcb <pid>：需确认进程彻底消失，且内存资源被正确回收。
3. block_pcb <pid>：进程状态转为BLOCKED并移出调度队列。
4. wakeup_pcb <pid>：状态恢复为READY，重新进入就绪队列等待调度。
5. show_pcb <pid>：格式化输出进程完整快照，含优先级、内存占用等关键元数据。

⚠ 现场随机抽查机制

验收时，老师将从10个命令中随机抽取5个进行现场实操测试。这意味着仅完成部分命令无法保证通过，必须确保全量功能均已开发完毕且无基础Bug。建议提前进行全覆盖自测，避免因个别命令未完成影响整体评分。

视图与调度组：

6. list_pcb：简洁列表形式呈现所有活跃进程关键状态。
7. ptree：必须输出层级分明的树形结构，清晰展示父子进程血缘关系。
8. suspend <pid>：进程挂起，保留状态但退出调度序列。
9. resume <pid>：恢复挂起进程，使其重新参与CPU竞争。
10. renice：动态调整优先级后，需验证队列排序位置实时更新。

✖ 核心红线：一票否决规则

若被抽查的5个命令中，任意一个未实现或存在严重逻辑错误（如程序崩溃、数据严重失真），则该模块20分将直接扣除10分！特别是`ptree`树形结构、内存回收等核心项，是老师重点关注的扣分点，请务必作为开发优先级最高的任务完成。

调度器——如何“动”起来

核心规则：多级反馈队列机制

定义三级队列Q0/Q1/Q2，时间片按2→4→8倍增。新进程默认入Q0，时间片耗尽则降级至下一级；阻塞唤醒后需按策略回归原级或重置Q0。调度时严格遵循“高优先级队列优先”，仅当高优先级队列为空时才调度低优先级队列，形成动态的优先级反馈闭环。

关键命令：8分功能验收点

start_sched (2分): 启动后台调度线程，终端必须实时输出进程分配、时间片消耗的调度日志流。

stop_sched (2分): 安全暂停调度，精准冻结当前所有进程、队列状态与时间片剩余值，支持断点恢复。

step (2分): 单步执行核心，必须完整打印队列快照、选程依据、执行时长及最终状态变更。

restart_sched (2分): 重启调度

演示重点：动态效果呈现

构建3个CPU密集型测试进程，执行start_sched后，需直观展示进程在不同队列间的升降级与交替运行。执行step时，必须现场展示“队列当前状态→调度决策逻辑→执行结果”的完整文本链路，让老师清晰看到调度器的动态计算过程而非静态数据修改。

⚠️ 核心红线：拒绝“假调度”，必须实现真实的动态执行！

本模块是课设的核心考核点，若仅在内存中静态修改进程状态数值，而无真实的时间片推进、无终端的实时日志输出、无step命令的决策推导过程，将直接判定为功能未实现（记0分）。验收的关键是让老师看到“进程在跑，调度在算”的动态行为，而非死的状态变更。

内存管理——动态分区模拟

核心验收：8项内存操作命令考核标准（共16分）

alloc <size>
功能：动态申请内存空间

验收：需正确返回分配起始地址；若空间不足，输出失败提示信息。

free_mem <addr>
功能：释放指定地址内存

验收：执行后调用show mem，验证对应内存块已标记为空闲状态。

show_mem
功能：可视化内存布局

验收：清晰展示已分配块和空闲块的链表结构，包含地址与大小信息。

compact
功能：内存碎片紧缩

验收：将所有离散空闲区合并，形成单一的大空闲块，消除外部碎片。

mem_stat
功能：内存使用统计

验收：准确输出总内存、已用空间、空闲空间及当前内存碎片率数值。

set_alloc_algo
功能：切换分配策略

验收：支持FF/BF/WF切换，后续alloc操作需严格遵循对应算法逻辑。

pgfault
功能：模拟缺页中断

验收：触发后输出标准缺页提示，体现虚拟内存的异常处理机制。

swap_out <pid>
功能：进程内存换出

验收：执行后show mem中该进程块消失，模拟交换空间的写入操作。

关键演示：制造内存碎片

多次alloc后释放中间块，调用mem_stat验证碎片率>0，直观展示外部碎片产生过程。

关键演示：执行紧缩操作

执行compact命令后，再次查看mem_stat，需证明碎片率降为0，空闲区连续。

关键演示：算法差异验证

切换FF/BF/WF三种算法，相同申请场景下，观察alloc选择的空闲块不同。

持久化——静态保存与恢复（20分）

save 命令（核心：全量快照）

将当前操作系统内核的全部运行状态，包括进程、内存、调度等，完整序列化并写入**二进制物理文件**中，形成系统的“静态镜像”，需确保数据无遗漏、无错误。

load 命令（核心：环境复原）

程序启动时自动触发的关键流程，从指定二进制文件中反序列化数据，将内核恢复至执行 save 时的**精确运行状态**，是验证持久化功能正确性的核心入口。

进程信息 PCB

必须完整包含所有进程控制块（PCB）的字段内容，如 PID、状态、优先级、资源占用标记等元数据。

进程树结构

准确记录进程间的父子层级与依赖关系，恢复后进程树的拓扑结构需与保存前完全一致。

多级调度队列

保存各优先级队列的顺序、队列内进程的排列次序以及调度器当前的上下文指针。

内存分区表

包含空闲分区链表、已分配内存块的映射关系及内存分配算法的当前执行状态。

关键动态数据

验收核心！必须保存进程已执行CPU时间、在队列中的精确位置，这是判定恢复精度的关键。

STEP 1 · 造状态

创建多进程，运行调度算法并分配内存，人为构建非空、非初始的复杂内核运行环境。

STEP 2 · 执行Save

输入 save 指令，触发内核状态的全量序列化逻辑，生成并落地二进制持久化文件。

STEP 3 · 彻底退出

关闭所有终端窗口与程序进程，确保内核完全终止，模拟真实的系统停机或异常崩溃场景。

STEP 4 · 重启Load

重新启动内核程序，系统自动加载外部二进制文件，执行数据解析与状态逆向恢复。

STEP 5 · 结果核验

执行 list_pcb、show_mem 等调试命令，确认所有状态与退出前完全一致，通过即得分。

多线程并发设计（18分）

前台交互线程：输入与封装

负责读取用户在命令行的输入指令，将复杂的命令参数封装为统一的消息对象，随后将消息安全投递至全局共享消息队列，完成交互层与核心业务逻辑的解耦。

后台维护线程：执行与处理

独立于前台的常驻工作线程，循环监听消息队列。取出任务后解析消息类型，执行进程创建、调度、销毁等底层操作，并将执行结果回写至共享状态区，实现异步处理。

同步互斥：临界资源保护

采用“消息队列 + 互斥锁/条件变量”的双重保障。消息队列实现生产消费解耦，互斥锁严格控制对PCB表、内存分配表等共享数据的并发访问，杜绝脏读与数据不一致。

实战演示：多实例并发共享（拉开分差的关键）

演示环境搭建：同时启动两个独立的命令行终端（实例A、B），确保二者挂载同一套底层共享数据文件/内存映射区。

关键验证动作：A端执行创建/调度命令，B端执行overview刷新，需实时看到新进程及状态变化，证明跨实例数据同步有效。

基础架构 · 10分（核心红线）

必须严格实现前后台线程物理分离与消息队列通信。若仅用单线程模拟多任务，或对临界资源未加锁保护，本项直接记0分。这是课设通过的基础门槛。

多实例共享 · 8分（能力体现）

完成跨终端的并发访问与状态一致性验证。这不仅是对多线程的考验，更是对进程间通信（IPC）机制设计的实战检验，是获得高分的重要差异化表现。

可视化展示——overview 命令 (10分)

进程树 (Process Tree)

以树形结构呈现进程间的父子派生关系，核心需包含PID、运行状态（RUNNING/READY等）、调度优先级及实时内存占用值，直观展现系统进程的层级架构与基础资源消耗。

内存ASCII图 (Memory Map)

采用字符画形式可视化内存空间，通过占位符与标识清晰区分已分配的进程内存块与空闲内存区域。需精准映射物理地址范围，让用户一眼读懂内存资源的分配与剩余情况。

多级反馈队列 (MLFQ)

按优先级分组展示各队列中的进程清单，明确标注不同队列的优先级区间。清晰呈现调度策略的执行结果，体现进程在不同优先级队列间的流转与当前归属。

参考输出示例：

```
Process Tree: init(1)[RUN] └─shell(2)[RDY] └─logger(3)[BLK]
Memory Map: |##init(64K)|--free(192K)--|##shell(128K)|--free(640K)--|
```

```
MLFQ Status:
Q0(p0-3): init | Q1(p4-7): shell | Q2(p8-15): logger
说明：高优队列无等待时，低优队列进程才会获得调度机会
```

关键验收：内容完整性

输出结果必须严格覆盖进程树、内存ASCII分布图、MLFQ队列状态这三个核心板块，缺一不可。这是系统全景状态展示的基础，也是验证系统模块整合度的首要标准。

关键验收：动态一致性

执行调度、进程创建/销毁等操作后，重新执行 overview 命令，显示的进程状态、内存分配及队列内容必须同步发生变化。静态不变的输出将被判定为功能未实现。

老师现场验收的典型场景

场景一：调度动态性

核心操作：创建3个CPU密集型进程，执行 start_sched 启动调度器，触发进程并发执行。

验收标准：日志持续滚动输出，清晰观察到进程的轮转调度、优先级降级及任务正常结束的全流程。

场景二：单步决策执行

核心操作：先用 stop_sched 暂停自动调度，随后多次执行 step 命令，驱动系统单步推进。

验收标准：每一步 step 均输出完整的内核决策过程，包括就绪队列扫描、优先级判定及上下文切换细节。

场景三：状态持久化

核心操作：构建多进程运行状态 → save 存储快照 → 退出程序后重启 → 执行 list_pcb 恢复数据。

验收标准：重启后所有进程状态完全恢复，包括队列顺序、剩余时间片及已执行时长等关键数据无损。

场景四：并发数据共享

核心操作：开启双终端窗口，窗口A动态创建新进程，窗口B持续执行 overview 总览命令。

验收标准：窗口B能毫秒级实时感知新进程的加入，数据无延迟同步，临界区资源访问无竞争错误。

场景五：内存碎片治理

核心操作：循环 alloc 申请小内存制造外部碎片 → 执行 compact 紧凑算法 → mem_stat 统计。

验收标准：紧凑后内存碎片率显著下降，空闲分区合并为连续大空间，内存分配算法切换逻辑生效。

场景六：终端可视化

核心操作：在系统运行多任务的复杂状态下，执行 overview 命令展示全局系统视图。

验收标准：运行、就绪、阻塞三区数据准确无误，表格排版整齐规范，关键元数据无乱码或错位。

评分标准总览（百分制）

账户管理 · 8分

完整实现账户登录、注销、密码错误锁定。

进程命令 · 20分

覆盖10个核心命令（各2分），随机抽测5个功能点。若有1个命令执行失败或结果错误，该部分扣10分。

调度器 · 8分

后台线程自动调度并生成日志；支持单步展示决策过程；高负载下的降级逻辑触发正确，无死锁。

内存命令 · 16分

完成8项内存操作（分配/释放/紧缩等），内存统计准确，置换算法切换生效，无内存泄漏。

数据持久化 · 20分

实现save/load机制，文件格式规范；系统重启后自动恢复队列顺序与动态数据，状态零丢失。

并发控制 · 18分

前后台线程分离，消息队列同步；多实例共享状态访问互斥，并发场景下数据一致性得到保证。

可视化 · 10分

Overview界面三大核心区域完整；数据实时动态更新，可视化效果直观反映内核运行状态。

项目总分 · 100分

需通过所有功能模块验收，各环节无重大缺陷。核心功能实现是及格基础，细节与稳定性决定高分。

启动阻断红线

程序无法启动或核心命令全失败，总分上限20分。基础运行能力是评分前提。

并发实现硬指标

单线程模拟多线程直接判0分。必须使用原生线程机制实现前后台任务并行处理。

调度器有效性

纯手工切换无自动运行，调度模块封顶4分。必须实现后台自动调度与策略执行。

报告30分+验收60+平时表现10分（中期验收，考勤等课堂表现）

避免踩的坑（扣分重灾区）

调度器“假动态”

仅修改状态字符串，未实现时间推进、队列降级与日志输出，核心调度逻辑未真正动态运行，属于形式化实现。

严重后果：调度器部分计 0 分

内存泄漏隐患

进程正常撤销后，其占用的内存资源未被及时释放回收，造成系统内存碎片与资源浪费，破坏内存管理闭环。

严重后果：进程+内存管理双重扣分

持久化“存一半”

仅保存PCB静态基础信息，关键的队列执行顺序、进程已执行时间等动态运行数据未落地，恢复时无法还原现场。

严重后果：直接扣除 5-8 分

多线程“有名无实”

未真正实现后台独立工作线程，或对共享临界区数据未加互斥锁保护，并发场景下存在数据不一致与执行错误。

严重后果：并发部分直接 0 分

可视化“偷工减料”

未按课设规范输出树形结构关系图与ASCII内存布局图，核心运行状态无法直观呈现，降低系统可验证性。

严重后果：一次性扣 5 分以上

报告“缺斤少两”

实验报告中遗漏自定义文件格式说明、未绘制核心执行流程图，文档完整性与可读性不满足验收标准。

严重后果：报告项直接扣 10 分

时间安排（32学时）

需求分析与设计	编码实现	调试与验证	报告与验收
第 1 - 8 学时	第 9 - 20 学时	第 21 - 28 学时	第 29 - 32 学时
明确方案 · 架构先行 深度理解课设业务目标，完成核心数据结构的定义与业务逻辑流程的可视化绘制。同时规范数据文件的存储格式，这是后续开发的基石，务必在本阶段确认所有设计细节，避免返工。	核心开发 · 功能落地 依据设计文档完成代码编写，重点实现用户命令接口、任务调度核心算法、内存资源的动态管理模块以及数据的持久化存储功能。在编码中遵循规范，保证模块间的解耦与代码的可维护性，确保各功能模块独立运行正常。	问题排查 · 场景测试 重点攻克多线程环境下的并发问题与死锁风险，利用调试工具定位逻辑错误。通过模拟多种边界场景和验收标准进行全流程黑盒测试，验证系统在高负载及异常输入下的鲁棒性，确保系统功能与设计预期完全一致。	成果沉淀 · 最终交付 整理开发过程中的技术文档，撰写包含设计思路、实现细节、测试结果的课程设计报告。准备最终的演示环境与验收材料，完成现场功能演示，清晰阐述技术选型与解决的关键问题，顺利通过最终项目验收。

提交材料清单

打包命名硬性规范：所有材料需统一压缩为 ZIP 格式，文件名严格遵循“学号_姓名_OS课程设计.zip”。请务必检查命名格式，避免因命名错误导致材料无法正常归档与审核。

核心源代码文件

提交项目全部功能实现的源文件（.c / .cpp）。要求对进程控制、内存管理、文件操作等关键逻辑添加详细中文注释，清晰展现代码设计思路与底层实现逻辑。

课程设计报告文档

采用 Word 或 PDF 格式提交。需包含系统设计蓝图、核心数据结构定义、业务流程可视化图表、关键代码解析及完整测试验证过程，形成逻辑闭环的技术设计文档。

可运行发行程序包

提供无需二次编译的可执行文件（.exe），必须附带程序运行所需的初始空物理文件。确保评审环节可直接运行演示，直观展示系统功能完整性与运行效果。

报告核心要素清单

需涵盖设计目标与整体思路；明确PCB、就绪/阻塞队列、内存表等数据结构定义；绘制清晰的进程调度与内存分配流程图，用可视化方式呈现系统核心架构。

验收验证与复盘总结

需包含关键代码功能解释、对应验收场景的实际运行截图；同时复盘开发难点（如死锁规避、内存碎片处理），详细描述问题排查过程与最终解决方案。

纪律要求

严禁抄袭

课程设计期间，代码雷同或实习报告抄袭行为将被视为严重学术不端。一经发现，相关作业或项目一律按零分处理，并立即上报学院，按学校学术规范进行后续纪律处分，绝不姑息。

考勤管理

考勤结果直接决定实习有效性：缺席两次实习课程将无最终成绩；缺席一次扣20分；迟到或早退一次扣10分。请务必按时到场，确有特殊情况需提前向指导老师履行请假手续。

秩序规范

在实验室及实习场地内，需保持安静专注的学习环境。严禁玩游戏、大声喧哗、随意走动等扰乱秩序的行为。每违反一次纪律扣5分，多次违反且不听劝阻者，将被取消本次实习资格。

重要提示：请各位同学严格遵守上述纪律条款，考勤表现与纪律执行情况将按比例计入课程设计最终成绩。学术诚信是立身之本，良好的实习秩序是任务完成的保障，希望大家端正态度，认真对待，顺利完成本次课程设计的各项任务。

Q & A

Q: 可以用 Python 进行开发吗?

只能 C/C++。Python 的 GIL 机制可能影响多线程并发性能，且二进制文件处理流程相对繁琐。

Q: 调度时间片必须真等2秒吗?

开发阶段可以使用 sleep(2) 模拟时间片轮转逻辑。在最终验收演示时，为了节省时间，允许将等待时间调快（如缩短为0.5秒或直接跳过），只要核心的调度算法逻辑正确、状态切换符合设计要求即可。

Q: 内存管理的大小单位是什么?

建议统一以 KB（千字节）作为内存分配与管理的基础单位。总内存容量可由开发者自行设定，通常推荐设置为 1024KB（即1MB），方便计算和测试。需保证内存块的申请、释放、碎片整理等操作均基于此单位正确执行。

Q: 物理文件一定要二进制格式吗?

这是硬性要求：**必须使用二进制文件**。如果使用文本文件存储数据，将被判定为不合规，在“持久化存储”部分会扣除10分。二进制格式能更真实地模拟磁盘数据存储方式，也能更好地处理复杂数据结构的读写。

Q: 多实例共享物理文件如何实现?

推荐简单高效方案：后台启动守护线程定期检查物理文件的修改时间戳，若检测到文件内容发生变化则自动重新加载数据。同时，引入文件锁机制（如独占锁），确保同一时刻只有一个实例拥有写权限，避免多进程同时写入造成的数据错乱问题。